

understand what changed and how to recover. This workflow was important because our group had many features being developed at the same time, and organizing the work helped us avoid unnecessary conflicts.

Improvements and Additional Features

If I had more time, I would first improve our handling of group state. Conditional rendering worked well for the Home page, but over time Chores, Expenses, and Calendar also became dependent on whether a user belonged to a group. A GroupContext or GroupProtectedRoute would have made that dependency clearer and would have reduced repeated fetches and repeated conditional logic.

I would also improve our developer workflow. Docker helped us run the app consistently, but a Makefile would have made common commands much easier to remember. Commands such as make up, make seed, make reset-db, make test, and make logs would have made the project friendlier for new teammates and would have matched the build automation ideas from lecture. I would also add database migrations through Flask-Migrate or Alembic so schema changes were tracked instead of relying on resets.

The final improvement I would make is automated testing. My seed users gave us repeatable manual testing, but they did not protect us from regressions by themselves. I would add tests for login redirects, route guards, create group, join group, leave group, no-group Home rendering, and dashboard stats. On the product side, I would also add notifications for upcoming chores, better expense settlement history, and a clearer onboarding flow for join codes.

Mistakes / What I Learned

The biggest mistake I made, in my opinion, was assuming that environment setup would be a small problem. At the start, it felt like each person could simply install the dependencies and run the frontend and backend locally. However, this quickly became one of the most frustrating parts of development. Different Python versions, Node versions, ports, and local database setups caused bugs that looked like application bugs, even when the actual code was fine. This wasted time because we were debugging machines instead of debugging Homiely.

The reason this went wrong was that we treated setup as an informal process instead of a real part of the software system. It should have been handled earlier with a clearer Docker workflow, better documentation, and preferably a Makefile. If we had written down the exact commands for starting the app, seeding the database, resetting the database, and checking logs, we would have saved a lot of time. This mistake taught me that build and setup instructions are not side details; they are part of the project.

Another mistake was relying too much on manual testing for too long. The demo users and DEMO123 group helped a lot, but they were only a baseline. They made bugs reproducible, but they did not automatically tell us when we broke something. The better approach would have been to turn those repeated manual checks into automated tests for login redirects, group creation, joining groups, leaving groups, and no-group Home rendering. This would have given us more confidence when merging changes.

I also learned that a design can be reasonable early and still become messy later. Conditional rendering on Home was the right choice when I first implemented it because it was simple and intuitive.